C. Garland

Codeforces Round 398 (Div. 2) · Problem 767C · Trees / Greedy / DFS

Problem statement ■ · Author repository ■

The problem

We are given a tree whose vertices have integer weights, possibly negative. Remove two edges so that the tree becomes three non-empty connected components with equal sums.

Let T be the sum of all weights. T must be divisible by 3; otherwise the answer is –1. Set S = T / 3. We need two cut positions that each close a component of sum S.

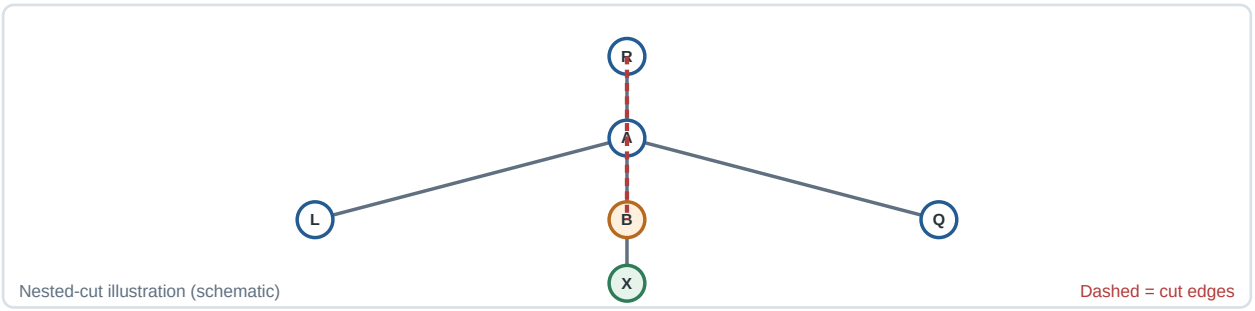
My key observation

I use postorder DFS. At each vertex, I calculate the sum of the part of its subtree that has **not already been separated**. If that remaining sum becomes S, I select the edge above this vertex and return 0 to its parent.

Why return 0?

That component has already been cut off. Returning its original sum would make an ancestor count those vertices again. Returning 0 excludes the completed component from all later subtree calculations.

Visual intuition: nested cuts



The two cuts may lie on the same root-to-leaf path. The upper cut then separates the remaining middle region after the lower component has been excluded. The zero return makes this accounting work.

Author: Richi Ahmed · **Collection:** Algorithms-and-Proofs

Algorithm and correctness

1. Root the tree

The vertex whose parent is 0 is the root. Build an undirected adjacency list and run DFS from it.

2. Postorder DFS

At u, start with weight[u] and add the values returned by its children. If the remaining sum equals S and u is not the root, record u as a cut vertex and return 0. Otherwise return the sum.

3. Answer

If two cut vertices are found, output their IDs; the edges to remove are their parent edges. Otherwise output –1.

Correctness proof

Lemma 1. Whenever u is selected, the still-unseparated connected region containing u has sum S. DFS processes children first, and each child's return value excludes components already cut off below it.

Lemma 2. A component already selected is never counted again. The selected vertex returns 0, so every ancestor excludes it. For nested cuts, a later cut closes the remaining region above the earlier one, not the earlier component itself.

Theorem. Two selected cut positions produce three non-empty components of sum S. The two closed components each have sum S and are distinct. The root-side remainder is non-empty because the root is never selected. Its sum is $T - S - S = 3S - 2S = S$.

Complexity

Time: O(n), because each vertex and edge is processed a constant number of times. **Space:** O(n) for the adjacency list, weights, and DFS.

Accepted C++ implementation

Cleaned-up transcription of the accepted submission. Each answer vertex identifies the edge between that vertex and its parent.

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

void solve() {
    int n;
    cin >> n;

    vector<vector<int>> adj(n + 1);
    vector<int> val(n + 1);
    int total = 0, root = -1;

    for (int i = 1; i <= n; i++) {
        int p, v;
        cin >> p >> v;
        val[i] = v;
        total += v;

        adj[p].push_back(i);
        if (p != 0) adj[i].push_back(p);
        else root = i;
    }

    if (total % 3 != 0) {
        cout << -1 << endl;
        return;
    }

    int target = total / 3;
    int ans1 = -1, ans2 = -1;

    auto dfs = [&](auto&& dfs, int u, int parent) -> int {
        int sum = val[u];

        for (int v : adj[u]) {
            if (v != parent)
                sum += dfs(dfs, v, u);
        }

        if (sum == target && u != root) {
            if (ans1 == -1) ans1 = u;
            else if (ans2 == -1) ans2 = u;
            return 0; // This component is already separated.
        }

        return sum;
    };

    dfs(dfs, root, 0);

    if (ans2 == -1) cout << -1 << endl;
    else cout << ans1 << ' ' << ans2 << endl;
}

int32_t main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    solve();
}
```

Implementation notes

• Parent = 0 identifies the root.
• The adjacency list stores edges in both directions; DFS skips the parent.
• `#define int long long` protects total and subtree sums.
• This is recursive DFS. With n up to 10■, a path-shaped tree can cause deep recursion; iterative postorder DFS is safer if stack limits become an issue.

Codeforces 767C ■ · lima-nine9/Algorithms-and-Proofs ■